SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Assignment — Slot B — Solution

Field	Details
Programme	B.E / B.Tech
Course Code	CSA1707
Course Name	ARTIFICIAL INTELLIGENCE
Max Marks	100
CO / Bloom's Level	CO1 / BL3
Title	Develop an Intelligent Student Academic Advisory Agent Using First-Order Logic
Team Members	V. Lasya - 192524186 , S.Divya sri – 192524388, V. G . Abbiramy - 192524097

Problem Statement

Design and develop a small Intelligent Student Academic Advisory Agent using First-Order Logic (FOL). The agent represents student academic information — attendance, examination performance, assignment status, subject performance, programming skills, academic strengths and weaknesses — and uses logical reasoning (unification, forward chaining, backward chaining and resolution) to derive academic recommendations such as the need for remedial classes, faculty interaction, or eligibility for advanced learning activities.

1. Knowledge Representation

The knowledge base models three students (Ravi, Meena, Arjun) across two subjects (AI, DBMS) using the predicates below, followed by the ground facts.

1.1 Predicates

Predicate	Meaning
Student(x)	x is a student.
Subject(s)	s is a subject.
Enrolled(x, s)	Student x is enrolled in subject s.
Marks(x, s, m)	Student x scored m marks (out of 100) in subject s.
Attendance(x, p)	Student x has p% overall attendance.
AssignmentSubmitted(x, s)	Student x has submitted the assignment for subject s.
AssignmentPending(x, s)	Student x has a pending assignment for subject s.
ProgrammingSkill(x, level)	Student x has programming skill of the given level (Weak / Average / Strong).

Predicate	Meaning
Strength(x, area)	x has an academic strength in area.
Weakness(x, area)	x has an academic weakness in area.
NeedsRemedial(x, s)	[Derived] x needs a remedial class in subject s.
NeedsSupport(x)	[Derived] x needs general academic support.
LowAttendance(x)	[Derived] x has attendance below the acceptable threshold.
RequiresFacultyInteraction(x)	[Derived] x should be counselled by faculty.
EligibleAdvanced(x)	[Derived] x is eligible for advanced learning activities.
Mentor(f, x)	[Derived] faculty f is assigned as a mentor to student x.
RemedialClassRequired(x, s)	[Derived] a remedial class must be scheduled for x in subject s.

1.2 Facts (Knowledge Base)

Twenty ground facts are asserted — well above the minimum of 8 — so that the rules in Section 2 have enough data to demonstrate every reasoning technique required.

F1. Student(Ravi)
F2. Student(Meena)
F3. Student(Arjun)
F4. F1Subject(AI)
F5. Subject(DBMS)
F6. Enrolled(Ravi, AI)
F7. Enrolled(Meena, DBMS)
F8. Enrolled(Arjun, AI)
F9. Marks(Ravi, AI, 38)

F10. Marks(Meena, DBMS, 82)

F11. Marks(Arjun, AI, 55)

F12. Attendance(Ravi, 62)

F13. Attendance(Meena, 91)

F14. Attendance(Arjun, 74)

F15. AssignmentPending(Ravi, AI)

F16. AssignmentSubmitted(Meena, DBMS)

F17. ProgrammingSkill(Arjun, Strong)

F18. ProgrammingSkill(Ravi, Weak)

F19. Strength (Meena, DBMS)

F20. Weakness(Ravi, AI)

2. Logical Rules

Eight FOL rules are constructed (minimum required: 6). Each rule generalises over students, subjects and marks using variables x, s, m, p and f.

R1. Remedial need — a student scoring below 40 in a subject needs a remedial class in it.

$$\forall x\ \forall s\ \forall m\ \ [Enrolled(x,s) \land Marks(x,s,m) \land (m < 40)]\ \Rightarrow\ NeedsRemedial(x,s)$$

R2. Academic support from remedial need — needing remedial help in any subject implies needing general support.

$$\forall x\ \forall s\ \ [NeedsRemedial(x,s)]\ \Rightarrow\ NeedsSupport(x)$$

R3. Low attendance — attendance below 65% is classified as low attendance.

$$\forall x\ \forall p\ \ [Attendance(x,p) \land (p < 65)]\ \Rightarrow\ LowAttendance(x)$$

R4. Faculty interaction from low attendance.

$$\forall x\ \ [LowAttendance(x)]\ \Rightarrow\ RequiresFacultyInteraction(x)$$

R5. Academic support from a pending assignment.

$$\forall x\ \forall s\ \ [AssignmentPending(x,s)]\ \Rightarrow\ NeedsSupport(x)$$

R6. Eligibility for advanced learning activities — high marks and acceptable attendance.

$$\forall x\ \forall s\ \forall m\ \ [Enrolled(x,s) \land Marks(x,s,m) \land (m \geq 80) \land \neg LowAttendance(x)]\ \Rightarrow\ EligibleAdvanced(x)$$

R7. Mentor assignment — every advanced-eligible student is guaranteed at least one mentor (existential quantifier nested inside a universal rule).

$$\forall x\ \ [EligibleAdvanced(x)]\ \Rightarrow\ \exists f\ \ Mentor(f,x)$$

R8. Remedial class scheduling — combines a remedial need with a pending assignment in the same subject.

$$\forall x\ \forall s\ \ [NeedsRemedial(x,s) \land AssignmentPending(x,s)]\ \Rightarrow\ RemedialClassRequired(x,s)$$

3. Variables and Quantifiers

Five variables are used across the rule base — x (student), s (subject), m (marks), p (attendance percentage) and f (faculty) — well above the minimum of 2.

3.1 Universal Quantification (∀)

Every rule in Section 2 is universally quantified so that it applies to any student/subject matching the pattern, not just the named individuals in the facts. For example:

$$\forall x\ \forall s\ \forall m\ \ [Enrolled(x,s) \land Marks(x,s,m) \land (m < 40)] \ \Rightarrow\ NeedsRemedial(x,s)$$

"For every student x, every subject s and every mark m: if x is enrolled in s and scored m in s and m is below 40, then x needs a remedial class in s." This single statement covers Ravi, Meena, Arjun and any future student added to the knowledge base.

3.2 Existential Quantification (∃)

Rule R7 demonstrates existential quantification, asserting that a mentor exists without naming one:

$$\forall x\ \ [EligibleAdvanced(x)] \ \Rightarrow\ \exists f\ \ Mentor(f,x)$$

"For every student x who is eligible for advanced learning, there exists at least one faculty member f who mentors x." The agent does not need to know which specific faculty member in advance — only that a suitable mentor can be found — which is exactly the kind of "some individual exists" statement propositional logic cannot express.

4. Queries

Two queries are posed. Neither answer is an explicitly stated fact — both require chaining through the rules in Section 2.

Query 1

```
NeedsSupport(Ravi) ?
```

Not stated directly. Derivable two independent ways: via Marks(Ravi,AI,38) → NeedsRemedial(Ravi,AI) → NeedsSupport(Ravi) [R1, R2], and via AssignmentPending(Ravi,AI) → NeedsSupport(Ravi) [R5]. Answer: TRUE (see Section 6, Forward Chaining).

Query 2

```
EligibleAdvanced(Meena) ?
```

Not stated directly. Requires combining Enrolled(Meena,DBMS), Marks(Meena,DBMS,82) and the absence of LowAttendance(Meena) (from Attendance(Meena,91)) through rule R6. Answer: TRUE (see Section 7, Backward Chaining).

5. Unification

Unification finds a substitution θ that makes a rule's variables match specific facts. Applying rule R1 to Ravi's records:

```
Rule R1 antecedent:   Enrolled(x,s) ∧ Marks(x,s,m) ∧ (m < 40)
Facts available:      Enrolled(Ravi,AI)     Marks(Ravi,AI,38)
```

Step 1 — unify Enrolled(x,s) with Enrolled(Ravi,AI):

```
θ1 = { x / Ravi,  s / AI }
```

Step 2 — apply θ1 to Marks(x,s,m), giving Marks(Ravi,AI,m), and unify with the fact Marks(Ravi,AI,38):

```
θ2 = { m / 38 }
```

Step 3 — combine into the most general unifier (MGU) and substitute into the full antecedent:

```
θ = { x / Ravi,  s / AI,  m / 38 }
Enrolled(Ravi,AI) ∧ Marks(Ravi,AI,38) ∧ (38 < 40)   →  all conjuncts TRUE
```

Since every conjunct holds under θ, R1 fires and θ is applied to the consequent NeedsRemedial(x,s), producing the new fact:

```
NeedsRemedial(Ravi, AI)
```

6. Forward Chaining — Query 1: NeedsSupport(Ravi)?

Forward chaining starts from the known facts and repeatedly fires rules whose antecedents are satisfied, until the goal is produced.

```
Known facts:  Enrolled(Ravi,AI), Marks(Ravi,AI,38), Attendance(Ravi,62),
AssignmentPending(Ravi,AI)


Step 1: Apply R1 with θ={x/Ravi, s/AI, m/38}   (38<40 holds)
```

```
    => derive  NeedsRemedial(Ravi, AI)

Step 2: Apply R2 with θ={x/Ravi, s/AI}
    => derive  NeedsSupport(Ravi)              <-- GOAL REACHED

Step 3 (independent path): Apply R5 with θ={x/Ravi, s/AI} on
AssignmentPending(Ravi,AI)
    => derive  NeedsSupport(Ravi)   (confirms Step 2)

Step 4 (additional facts derived along the way):
    Apply R3 with θ={x/Ravi, p/62}  (62<65 holds) => derive LowAttendance(Ravi)
    Apply R4 with θ={x/Ravi}                 => derive
RequiresFacultyInteraction(Ravi)
```

Conclusion: NeedsSupport(Ravi) = TRUE, reached by forward chaining through two independent inference paths (R1→R2 and R5), which also surfaces the additional recommendation RequiresFacultyInteraction(Ravi).

7. Backward Chaining — Query 2: EligibleAdvanced(Meena)?

Backward chaining starts from the goal and works backward, generating subgoals until each is matched against a known fact.

```
GOAL: EligibleAdvanced(Meena)

Step 1: Find a rule whose consequent matches the goal -> R6
        Enrolled(x,s) ∧ Marks(x,s,m) ∧ (m≥80) ∧ ¬LowAttendance(x) =>
EligibleAdvanced(x)
        Unify goal with consequent: θ = { x / Meena }

Step 2: New subgoals (with x=Meena):
        (a) Enrolled(Meena, s)
        (b) Marks(Meena, s, m) with m ≥ 80
        (c) ¬LowAttendance(Meena)

Step 3: Solve (a) - matches fact Enrolled(Meena,DBMS)  =>  θ += { s / DBMS }

Step 4: Solve (b) - matches fact Marks(Meena,DBMS,82)  =>  θ += { m / 82 };  check 82
≥ 80 -> TRUE

Step 5: Solve (c) - try to prove LowAttendance(Meena) via R3:
        Attendance(Meena,p) ∧ (p<65) => LowAttendance(Meena)
        Fact: Attendance(Meena,91); check 91<65 -> FALSE
        R3 cannot fire, so LowAttendance(Meena) is NOT provable
        => ¬LowAttendance(Meena) holds
```

Step 6: All subgoals (a),(b),(c) satisfied under θ = {x/Meena, s/DBMS, m/82}

=> GOAL PROVED: EligibleAdvanced(Meena)

Conclusion: EligibleAdvanced(Meena) = TRUE, proved by backward chaining by reducing the goal to three subgoals that are each matched against the fact base.

8. Resolution — Proving NeedsSupport(Ravi)

To use resolution, the relevant facts and the rules R1/R2 are grounded with θ={x/Ravi, s/AI, m/38} (the arithmetic test 38<40 has already been evaluated as true, so it is dropped from the clause) and converted to clause form. The goal is negated and added to the clause set; resolution then derives the empty clause, proving the goal by contradiction.

8.1 Clause Form

C1: Enrolled(Ravi,AI) [fact]
C2: Marks(Ravi,AI,38) [fact]
C3: ¬Enrolled(Ravi,AI) ∨ ¬Marks(Ravi,AI,38) ∨ NeedsRemedial(Ravi,AI) [R1, grounded]
C4: ¬NeedsRemedial(Ravi,AI) ∨ NeedsSupport(Ravi) [R2, grounded]
C5: ¬NeedsSupport(Ravi) [negated goal]

8.2 Resolution Steps

Step 1: Resolve C1 & C3 on Enrolled(Ravi,AI)
=> C6: ¬Marks(Ravi,AI,38) ∨ NeedsRemedial(Ravi,AI)

Step 2: Resolve C6 & C2 on Marks(Ravi,AI,38)
=> C7: NeedsRemedial(Ravi,AI)

Step 3: Resolve C7 & C4 on NeedsRemedial(Ravi,AI)
=> C8: NeedsSupport (Ravi)

Step 4: Resolve C8 & C5 on NeedsSupport(Ravi)
=> C9: ∅ (empty clause -- contradiction)

Because adding ¬NeedsSupport(Ravi) to the knowledge base leads to a contradiction (the empty clause), the knowledge base logically entails NeedsSupport(Ravi). This confirms, via a third and independent reasoning method, the same conclusion reached by forward chaining in Section 6.

9. Comparison with Propositional Logic

If the same advisory agent were built in propositional logic, every student/subject/condition combination would need its own atomic symbol (e.g. Ravi_Needs_Remedial_AI, Meena_Needs_Remedial_DBMS, …) instead of one general rule. The table below compares the two approaches on the required dimensions.

Aspect	Propositional Logic	First-Order Logic (this agent)
Variables	None — every proposition is a fixed, ground symbol.	Uses variables x, s, m, p, f so one rule applies to any student/subject/mark.
Quantifiers	Not available — cannot express "all" or "some".	Supports ∀ (R1–R8) and ∃ (R7: ∃f Mentor(f,x)).
Representation of individuals	Students and subjects are not distinguishable objects, only opaque symbol names.	Ravi, Meena, AI, DBMS etc. are explicit terms/objects predicates can range over.
Relationships between entities	Cannot express structured relations; propositions are atomic.	Predicates such as Enrolled(x,s) and Marks(x,s,m) explicitly capture relationships.
Generalization of rules	A separate rule/proposition is needed per student-subject pair.	One rule (e.g. R1) generalises over every student and subject via variables.
Knowledge representation	Flat and unstructured; grows combinatorially and is hard to maintain.	Structured and compact; mirrors the real objects and relations in the domain.

Aspect	Propositional Logic	First-Order Logic (this agent)
Scalability	Adding a student/subject means adding many new propositions and rules.	Adding a student/subject only needs new ground facts — the rules stay the same.
Inference capabilities	Limited to propositional resolution/truth tables over fixed, ground symbols.	Supports unification, forward/backward chaining and resolution over variables, so it can reason about any student matching a pattern, including ones added later.

10. Discussion

10.1 How FOL Improves the Representation

FOL lets the advisory agent talk about objects (students, subjects, marks, faculty) and the relationships between them, rather than treating each condition as an isolated, unrelated symbol. A single rule such as R1 captures a policy that would otherwise need one hand-written proposition per student per subject, and quantifiers let the knowledge base state general policies ("every student below 40 needs remedial help") and existence claims ("an eligible student always has some mentor") that propositional logic simply cannot phrase.

10.2 Advantages

- Compactness — 8 general rules cover an unbounded number of students and subjects.
- Expressiveness — relations (Enrolled, Marks) and quantifiers (∀, ∃) model the domain naturally.
- Reusable, automatable inference — unification, forward/backward chaining and resolution all operate directly on the same rule set, as shown in Sections 5–8.
- Easier maintenance — adding a new student only means adding facts, not new logic.

10.3 Limitations

- General first-order inference is only semi-decidable — resolution is guaranteed to terminate when a fact follows from the KB, but may not terminate when it does not.
- Classical FOL has no built-in notion of uncertainty, so a student who is "usually" attentive or "borderline" on marks cannot be represented gracefully — this needs probabilistic or fuzzy extensions.
- Arithmetic comparisons ($m < 40$, $p < 65$) are not native FOL connectives; they must be supplied as evaluable predicates by the reasoning engine, as done implicitly in this design.

- Designing correct predicates and rules requires more upfront modelling effort than writing flat propositions.

10.4 Possible Extensions

- Add predicates for attendance/marks trends over multiple semesters to support temporal reasoning (e.g. Improving(x,s), Declining(x,s)).
- Add placement-readiness and extracurricular predicates to widen the scope of advice beyond academics.
- Integrate probabilistic or fuzzy logic to represent borderline cases (e.g. attendance of exactly 65%) instead of hard thresholds.
- Feed machine-learning-derived predicates (e.g. PredictedDropoutRisk(x)) into the same rule engine as additional facts, keeping the reasoning layer unchanged.
- Add an explanation module that reports the chain of rules used (as in Sections 6–8) so advisors can see why a recommendation was made.

Expected Deliverables — Coverage Summary

Deliverable	Where covered
FOL knowledge representation	Section 1
Minimum 8 facts	Section 1.2 — 20 facts (F1–F20)
Minimum 6 rules	Section 2 — 8 rules (R1–R8)
Minimum 2 variables	Section 3 — 5 variables (x, s, m, p, f)
Minimum 2 quantified statements	Section 3.1 (∀) and 3.2 (∃)
Minimum 2 logical queries	Section 4 — Query 1 and Query 2
Unification demonstration	Section 5
Forward-chaining demonstration	Section 6
Backward-chaining demonstration	Section 7
Resolution demonstration	Section 8
FOL vs. Propositional Logic comparison	Section 9
Discussion — advantages, limitations, extensions	Section 10

Individual Contribution

V.LASYA

I was responsible for the knowledge representation component of the assignment. This involved designing the predicates used to model student academic information — such as Student, Subject, Enrolled, Marks, Attendance, AssignmentPending, and

ProgrammingSkill — and building the complete knowledge base of 20 ground facts covering three students (Ravi, Meena, Arjun) across two subjects (AI, DBMS). I also worked on the variables and quantified statements section, identifying the five variables used across the rule base (x, s, m, p, f) and explaining the universal quantification used throughout the rules as well as the existential quantification demonstrated in the mentor-assignment rule. In addition, I prepared the comparison between First-Order Logic and Propositional Logic, evaluating both approaches across variables, quantifiers, representation of individuals, relationships between entities, generalization of rules, knowledge representation, scalability, and inference capabilities to show why FOL is better suited to this advisory agent.